server2 infrastructure preparation (S03)

Status: reproducible local test environment and approval-ready staging plan; no AWS resources or routing changed. S01/S04 are integrated. The foundation binary serves health and loopback metrics, without weather/S3/provider runtime routes. Architecture and S03 contract describe the boundaries.

Start and verify locally

Prerequisites: Python 3.11+, the pinned Rust 1.99 toolchain, and a built release binary. Run from server2/. Docker, MongoDB, SQLite, provider keys and AWS credentials are not required. Existing provider fixture bytes were copied once with provenance and hashes under config/fixtures/; the runtime never reads legacy files. These are repository oracle inputs, not new live measurements.

```
cargo build --locked --release
python3 deploy/local/test_local.py
python3 deploy/local/smoke.py --binary target/release/server2
```

The distinct smoke starts the foundation and two peers on OS-selected loopback ports, reads a recorded KMA response, gzip-encodes its exact bytes, tests conditional PUT/412 and HEAD/GET metadata, injects committed PUT response failure/loss, and reconciles the stored object. It then sends SIGTERM and verifies owned child/PIDs, all listeners and the ready file are gone. Unknown endpoints or credentials are rejected locally. No peer ever forwards a request to AWS or providers.

For an interactive session, create a private server2-s03-* directory in the system temporary directory and keep the stack in the foreground:

```
S03_DIR=$(mktemp -d "${TMPDIR:-/tmp}/server2-s03-XXXXXXXX")
python3 deploy/local/stack.py --binary target/release/server2 --ready "$S03_DIR/ready.json"
```

Read ready.json from a second terminal for the local endpoints and dummy keys. It becomes visible only after a complete JSON file is written and flushed in the same private directory, then atomically hard-linked with no replacement. The first open must parse complete JSON; do not retry JSON decode failures to hide a race. An existing file, symlink or dangling symlink is preserved and startup fails. Unsupported hard links fail closed; there is no overwrite/replace fallback. Ordinary write/publication failure cleans owned temporary files and processes. Normal stop removes the ready file only if its regular-file device/inode still matches this launch; a replacement entry is preserved.

The additional release-launcher observation check opens readiness immediately when available for 12 independent starts, without decode retries, and verifies health plus owned process, port, ready and temporary-file cleanup for each start:

```
python3 deploy/local/smoke.py --binary target/release/server2 --ready-observations 12
```

Ctrl-C/SIGTERM stops only the owned foundation child and peers. Remove the now empty task directory after stop. Ready paths must be absent, under a private task temporary directory; the launcher refuses external/non-owned binaries. Bind/startup failure exits nonzero and cleans owned children; stop does not contact production. Never use this test stack as a public server or point it at production credentials.

SIGKILL or a hard launcher crash bypasses cleanup: the Rust child may survive and ready.json or a private .ready-* temporary file may remain. A ready file is not proof of a live/current launcher. Before recovery, record its launcher/foundation PIDs and endpoints. Use a process listing (for example ps -p <foundation_pid> -o pid,ppid,lstart,command) to confirm the child is the exact owned release binary from this run, with the expected start time; never kill a PID solely because it appears in a stale ready file. Send SIGTERM only to that confirmed child, wait for exit, and confirm all recorded loopback listeners are closed. If identity cannot be established, stop and inspect manually. Only after confirming this run is stopped, remove its own stale ready file and this run's private temporary files/directory, then start with a fresh absent ready path. The launcher implements

normal-signal cleanup, not parent-death supervision.

Local S3 subset

Path-style /server2-local/key supports PUT/HEAD/GET; bucket GET with list-type=2 provides lexical prefix pagination. PUT requires Content-MD5 of compressed bytes and If-None-Match: * or If-Match: "etag". MD5 failure is 400; an existing raw key or failed CAS is 412. There is no deletion. HEAD/GET preserve exact compressed body length, Content-Type, ETag, version ID and x-amz-meta-*; an explicit versionId retrieves that retained local revision. Range, multipart and version listing are unsupported and rejected, rather than simulated. LIST accepts only list-type, prefix, max-keys and continuation-token (plus dummy signing fields); delimiter, start-after, fetch-owner, encoding-type and unknown options return 501. Duplicate query fields return 400. No Content-Encoding is added. The fixture provider accepts only recorded GET/HEAD paths.

The test header X-Server2-Test-Key: server2-local, or a presigned-query shape with dummy access key server2-local and a 64-hex signature, identifies a request. Dummy secret server2-local-secret and region us-east-1 are solely for SDK tests. The peer does not verify SigV4 signatures, IAM, TLS, clock/skew or AWS retries. Its finite volatile version map vanishes at process exit; it is not a server2 serving store, persistent emulator, MinIO replacement or AWS latency benchmark. Real AWS signing/conditional headers and role restrictions require S09 verification.

Default bounds: 1 MiB per body, 32 MiB accounted body/metadata/key/version overhead, 2 KiB user metadata (original UTF-8 wire key/value bytes, excluding the wire prefix), 8 KiB original request-header key/value bytes, 4,096 object keys, 64 versions/key and 16 handler connections. Both the per-operation socket timeout and absolute admitted-connection lifetime are 2 seconds; dripping header/body bytes cannot extend that lifetime. These bound local test growth; they are not an exact RSS promise or a claim to support every S3 object size. S05 production body caps are separate. The S3 metadata rules define the real service limits.

Authenticated test-only POST /__test/fault/error-after-put?key=<encoded-key> or /__test/fault/drop-after-put?key=<encoded-key> affects the next successful PUT for that key. Both commit first; the first returns 500, the second drops the response. Reconcile with HEAD/GET rather than assuming no object was stored. No fault endpoint exists in the production runtime.

Staging approval worksheet

The prerequisites JSON leaves actual account, region, bucket, role, owners and monthly ceiling unset. Fill them in a reviewed deployment plan before any resource operation. S03 does not contain an apply/provision command. Resource approval is separate from this PR or local start. Do not reuse the local dummy keys outside tests.

- Infrastructure owner confirms the origin region/account and private bucket in that same region, versioning Enabled, all Block Public Access settings, BucketOwnerEnforced ownership and default SSE-S3 encryption. No lifecycle deletion or transition is selected now. S09 measures version accumulation and retrieval costs; no Flexible/Deep Archive on synchronous serving paths.

- Runtime role gets only prefix-scoped ListBucket, GetObject and PutObject; historical index/state versions additionally use GetObjectVersion and scoped ListBucketVersions. HEAD uses GetObject authorization. There are no DeleteObject, delete-version, bucket-admin, EC2-write or broad AWS permissions in the role template. Placeholder substitution and IAM evaluation against real resources remain unexecuted. Encryption with customer KMS keys would require a new least-privilege review; it is not silently covered by this template. Unverified IAM risk: the ListBucket grant has an s3:prefix condition. AWS documents that missing-key reads return 404 when ListBucket is permitted, otherwise 403; whether this conditional grant suffices for HEAD/GET is not established by local tests. S09 must use the actual runtime role to HEAD and GET a known-absent raw

key and catalog key and require 404/NoSuchKey, while separately proving that unauthorized reads return 403. Record sanitized status, role/policy revision, region and time; a mismatch blocks readiness and requires reviewed policy/error-classification correction. Do not treat 403 as absence or widen permissions from this plan. See the GetObject missing-key rule.

- The bucket template rejects non-TLS and unconditional raw writes, and requires conditional catalog/ state publication. AWS supports conditional-write policy enforcement. The runtime role does not change versioning/policies. Operator approval covers installation and a deny-case test with the actual role, including attempted delete.

- Provider/quota owner: since S10e (#2753, AK 2026-10-15) server2 uses the legacy keys under their legacy names, from server2/.env, a private copy of the legacy server/.env: DONGNAE_SECRET_KEYS, AIRKOREA_SECRET_KEYS, VC_SECRET_KEY, WAQI_SECRET_KEY, OWM_SECRET_KEY, GOOGLE_SECRET_KEY, GEOCODER_KAKAO_KEYS, GEOCODER_GOOGLE_KEY, AWS_ACCESS_KEY, AWS_SECRET_KEY; the server2-only SERVER2_S3_BUCKET is added there. Server2 and legacy then share each key's provider quota. Values stay outside Git/logs. Confirm daily and peak calls/records for warm-up, cold geocoding, hourly/2-minute demand capture and required legacy collectors. Count data.go.kr calls and VC provider records, not logical requests; keep remaining legacy collection quota/cost visible.

- Operations owner confirms network ingress from the existing origin routing, loopback-only metrics, provider egress allowlist and a same-region TLS S3 path. Evaluate an S3 gateway endpoint and network charges with the actual topology. No gateway/Lambda/CloudFront changes are performed by S03.

Host, readiness and Spot interruption

Start with one Spot host and one Tokio multithreaded API process. ARM c7g.large or x86 c6i.large are measurement candidates, each 2 vCPU/4 GiB; there is no measured host choice or price claim in this PR. Availability, current Spot price and budget must be checked in the approved region before provisioning. Increase memory/CPU when S09 finds pressure; do not introduce multiple cache-owning API processes first.

S09 must build Rust 1.99 on the actual Linux target (aarch64-unknown-linux-musl or x86_64-unknown-linux-musl), verify release startup/signals/TLS CA/DNS and SDK behavior, and report RSS/CPU, cold p50/p95/p99, S3 throughput and provider budget headroom. Leave a proposed 25% host headroom until real workload sizing changes it. Root disk holds OS/binaries/sanitized logs only; there is no gp3 weather serving volume, SQLite, Redis or shared-instance cache.

/health proves process liveness only. Before a route receives traffic, later phases must prove configuration/keys, S3 role access, provider reservations, family readiness and bounded cold warm-up. The foundation does not yet implement that readiness gate or an IMDS watcher.

For a separately approved production drain, an IMDSv2 interruption/rebalance watcher marks the instance unready, removes new routing, stops new provider budget reservations/capture work, bounds completion of owned raw/fetch-group publications, then sends SIGTERM. Persisted-before-response acknowledgment is the durability boundary; the interruption signal is not. Spot notices are best effort, and hibernation does not provide the usual two-minute window. A rebalance recommendation may arrive alongside the interruption notice. Unexpected-loss testing therefore remains required even with a watcher. Local SIGTERM smoke is not an actual Spot test.

Rollback and later scale-out

Keep legacy and per-version family switches. /weather and /geocode move as a unit; a cold cell with unprovable history still forwards/retains legacy after a nominal family cutover. Rehearse CloudFront-aware rollback and count coexistence quotas. AK authorizes each operational cutover after goldens/shadow/performance; S03 does not retire MongoDB or promise cold-cell retirement is achievable.

D02/D03 prohibit automatic failed/unknown push resend and legacy registration migration. Preserve new accepted S3 registrations during rollback; do not broadcast on startup. Sanitize outcomes for operations to fix subsequent sends. Weather/provider retries remain their separate contract.

Future multiple instances keep independent memory caches and share S3. Before that rollout, implement durable provider admission, fetch/background ownership, CAS contention, readiness/drain and bounded warm-up. This is a plan only; no shared cache or multi-instance deployment is introduced.

Evidence and limitations

The selected CLI capture and manifest bind the actual test output and release binary. Editable source is this document; the PDF manual includes the same operational boundaries. Unit tests cover wire boundaries separately from integrated process smoke. No live AWS access, provider request, traffic replay, IAM application, musl Linux run, real Spot interruption, weather parity or latency/cost measurement was performed.

```
ACTUAL RELEASE-PROCESS SMOKE | 2026-10-15
python3.11 server2/deploy/local/smoke.py --binary target/release/server2

{
  "result": "PASS",
  "scenarios": [
    "complete ready JSON on first visible open",
    "health/metrics isolation",
    "recorded provider exact bytes",
    "conditional raw PUT and HEAD/GET",
    "unknown endpoint/credential rejection",
    "committed500/drop reconciliation",
    "owned SIGTERM cleanup"
  ],
  "raw_sha256": "7a272f41ac1a805851a8d6014f4450904ec8b2fb3228b1146c7ea1948f351a27",
  "gzip_bytes": 251,
  "drain_seconds": 1.061,
  "outbound_proxy": "absent",
  "aws_measurement": false
}
```